A aprendizagem de Máquina - II

Atividade para Compensação de Aula (19.09.2026)
Prof. Luis Claudio

ATENÇÃO:

Após a data limite, haverá desconto de uma falta por dia de atraso.

Situação-Problema 01 — Configuração e treinamento de uma MLP

Uma empresa deseja utilizar uma rede neural para classificar clientes entre duas categorias. Após a preparação dos dados, cada cliente é representado por **20 atributos numéricos**. A equipe construiu inicialmente uma MLP com duas camadas ocultas, contendo **64 e 32 neurônios**, ambas utilizando ReLU, e uma camada de saída com **1 neurônio e Sigmoid**.

O treinamento foi configurado com batch_size = 128, epochs = 40, learning_rate = 0,001 e otimizador Adam. Durante os testes, a equipe percebeu que diferentes configurações produzem resultados distintos e decidiu avaliar alterações tanto na arquitetura quanto no processo de treinamento.

Questões

1) Identifique, entre as configurações apresentadas, quais correspondem a **hiperparâmetros de arquitetura** e quais correspondem a **hiperparâmetros de treinamento**. Explique também por que pesos e biases não pertencem a nenhuma dessas duas categorias.

Resposta: Hiperparâmetros de arquitetura são as escolhas relacionadas à estrutura da rede, definidas antes do treinamento: número de camadas ocultas (2), quantidade de neurônios em cada camada (64 e 32), funções de ativação (ReLU e Sigmoid) e número de neurônios da camada de saída (1). Hiperparâmetros de treinamento controlam como ocorre o processo de otimização, sem alterar a estrutura da rede: batch_size (128), epochs (40), learning_rate (0,001) e o otimizador (Adam). Pesos e biases não pertencem a nenhuma das duas categorias porque não são escolhidos manualmente antes do treinamento: são parâmetros internos do modelo, inicializados e depois ajustados automaticamente pelo algoritmo de otimização (backpropagation e gradiente descendente) a cada iteração, com o objetivo de minimizar a função de perda.

2) A equipe pretende testar diferentes quantidades de camadas ocultas, neurônios e funções de ativação. Explique por que essas escolhas precisam ser avaliadas experimentalmente e como elas podem afetar a capacidade de representação da rede.

Resposta: Não existe uma fórmula fechada que determine, para um problema específico, o número ideal de camadas, neurônios ou a função de ativação mais adequada. Essas escolhas dependem da complexidade do problema, da quantidade e qualidade dos dados e do grau de não linearidade da relação entre entradas e saídas, fatores que só podem ser avaliados testando diferentes configurações e comparando o desempenho em dados de validação. Elas afetam diretamente a capacidade de representação do modelo: mais camadas e neurônios aumentam a capacidade de aproximar funções complexas, mas elevam o risco de overfitting e o custo computacional; poucas camadas ou neurônios podem gerar underfitting, por falta de capacidade. A função de ativação introduz a não linearidade necessária para representar relações complexas.

3) Considere agora alterações no batch_size, no número de épocas e no learning_rate. Explique como cada uma dessas alterações pode modificar o processo de treinamento, incluindo quantidade de atualizações, estabilidade do gradiente, velocidade de aprendizado e possibilidade de underfitting ou overfitting.

Resposta: Batch_size: lotes menores geram mais atualizações por época e uma estimativa de gradiente mais ruidosa (mais estocástica), o que pode ajudar a escapar de mínimos locais mas torna o treinamento menos estável; lotes maiores geram menos atualizações, com gradiente mais estável e suave, porém exigem mais memória e podem generalizar pior. Número de épocas: define o total de atualizações realizadas; poucas épocas podem causar underfitting, por o modelo não aprender os padrões, enquanto épocas em excesso podem causar overfitting, por memorização dos dados de treino. Learning_rate: controla o tamanho do passo na direção do gradiente; uma taxa muito alta causa instabilidade, oscilações e até divergência; uma taxa muito baixa torna o aprendizado lento, exigindo mais épocas e podendo travar em regiões subótimas.

4) Explique, de forma integrada, o papel da **função de perda, do gradiente, do learning rate e do otimizador** durante o treinamento. Deixe claro por que o gradiente não é um hiperparâmetro.

Resposta: A função de perda mede o quão distante as previsões estão dos valores reais, fornecendo um valor a ser minimizado. O gradiente é a derivada dessa função em relação a cada peso, indicando a direção e a intensidade do ajuste necessário para reduzir o erro. O learning_rate define a magnitude do passo dado nessa direção a cada atualização. O otimizador (Adam) usa os gradientes calculados — combinados com estimativas adaptativas de médias móveis, no caso do Adam — para atualizar os pesos de forma eficiente, acelerando e estabilizando a convergência. O gradiente não é um hiperparâmetro porque não é definido manualmente antes do treinamento: ele é calculado automaticamente a cada iteração, via backpropagation, a partir dos dados, da função de perda e dos pesos atuais, sendo portanto um resultado do processo de treinamento, assim como os pesos.

Situação-Problema 02 — Classificação de imagens utilizando CNN

Uma indústria deseja classificar imagens coloridas de produtos para identificar possíveis defeitos. Cada imagem possui dimensão **32 × 32 × 3**. A primeira camada convolucional utiliza **16 filtros 3 × 3**, stride 1, padding same e função ReLU. A rede possui posteriormente outra camada convolucional, uma operação de Max Pooling e, antes das camadas densas, uma operação Flatten.

Durante o desenvolvimento, alguns integrantes da equipe confundiram conceitos como canais, filtros, mapas de características e quantidade de parâmetros, além de questionarem por que utilizar uma CNN em vez de simplesmente transformar toda a imagem em um vetor e utilizar uma MLP.

Questões

5) Explique como um filtro 3 × 3 atua sobre uma imagem RGB. Indique a dimensão completa de cada filtro, como os três canais participam do cálculo e quantos mapas de características são produzidos pelos 16 filtros.

Resposta: Como a imagem tem 3 canais (RGB), cada filtro precisa ter profundidade 3 para operar sobre todos os canais ao mesmo tempo; a dimensão completa de cada filtro é, portanto, 3×3×3 (altura × largura × canais). No cálculo, o filtro é posicionado sobre uma região da imagem e é feita a multiplicação elemento a elemento com os pixels dos três canais; os resultados dos três canais são somados, junto com o bias, gerando um único valor escalar por posição. Repetindo esse processo por toda a imagem obtém-se um mapa de características por filtro; como há 16 filtros, são produzidos 16 mapas de características.

6) Calcule a quantidade de parâmetros treináveis dessa primeira camada convolucional. Em seguida, explique por que essa quantidade não aumenta quando a altura e a largura da imagem aumentam, desde que sejam mantidos a quantidade e o tamanho dos filtros e a quantidade de canais.

Resposta: Cada filtro tem 3×3×3 = 27 pesos mais 1 bias, totalizando 28 parâmetros por filtro; com 16 filtros, o total é (3×3×3 + 1) × 16 = 28 × 16 = 448 parâmetros treináveis. Essa quantidade não aumenta com a altura e a largura da imagem porque os pesos do filtro são compartilhados (parameter sharing): o mesmo conjunto de pesos desliza por toda a imagem, em vez de existir um conjunto de pesos por posição espacial, como em uma camada densa. O número de parâmetros depende apenas do tamanho do filtro, do número de canais de entrada e do número de filtros — não das dimensões espaciais da imagem.

7) Explique como **stride e padding** influenciam as dimensões espaciais dos mapas de características. Em sua resposta, diferencie same e valid.

Resposta: O stride define o passo com que o filtro se desloca sobre a imagem; um stride maior faz o filtro avançar mais posições a cada passo, reduzindo o tamanho espacial do mapa de características resultante, enquanto stride 1 preserva mais

detalhes espaciais. O padding é o preenchimento (geralmente com zeros) adicionado nas bordas da imagem antes da convolução. No modo valid, não há preenchimento, e o mapa de características resultante é sempre menor que a entrada, pois o filtro não ultrapassa os limites da imagem. No modo same, é adicionado padding suficiente para que a saída tenha as mesmas dimensões espaciais da entrada, quando o stride é 1.

8) A primeira camada produz 16 mapas de características, que são recebidos pela segunda camada convolucional. Explique qual será a **profundidade de cada filtro da segunda camada**, quantos mapas de características serão produzidos pelos seus 32 filtros e qual é a função do **Flatten** antes das camadas densas.

Resposta: Como a primeira camada produz 16 mapas de características, cada filtro da segunda camada precisa ter profundidade 16, para operar sobre todos os mapas recebidos, de forma equivalente ao que ocorre com os 3 canais RGB na primeira camada. Com 32 filtros, a segunda camada produzirá 32 mapas de características na saída — um para cada filtro. O Flatten converte a saída tridimensional das camadas convolucionais/pooling (altura × largura × profundidade) em um único vetor unidimensional, permitindo que essa informação seja usada como entrada para as camadas densas, que esperam vetores e não estruturas multidimensionais.